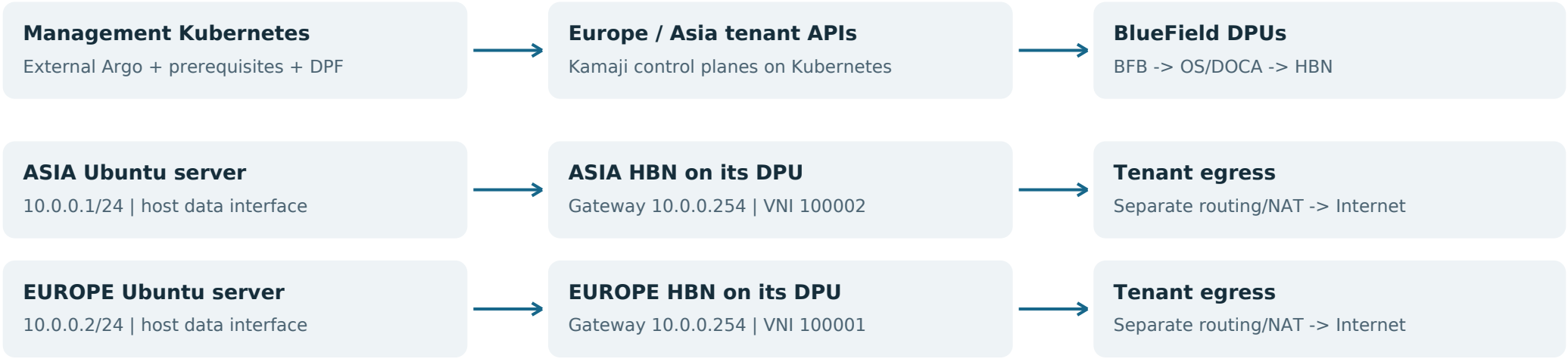
DPF only: install, connect, demonstrate

Student lab | DPF v26.4.0 | HBN 3.4.0 | BlueField-3 lab profile

Goal: use external Argo CD to install prerequisites and DPF, create Europe and Asia tenants, provision their DPUs, and examine Ubuntu 10.0.0.1 through HBN. No NICo installation is included.



Before you begin

Use an existing management Kubernetes cluster with Ready nodes, cluster-admin access, working DNS/NTP and Internet/registry access. Install kubectl, Helm, Bash, Python 3, curl, unzip and SSH on the administration machine.

This lab retains its existing DHCP, Ubuntu server installations, host netplan and switch/router configuration. Each tenant needs its own egress routing/NAT for the overlapping 10.0.0.0/24 range. These are prerequisites, not installed by this package.

For the existing working demo, start at Step 8. For installation, start at Step 1. Provisioning can replace the DPU OS and needs a planned host reboot. The profile is specific to the lab hardware; do not apply its firmware settings to BlueField-4.

External Argo (namespace argocd) manages the stack. DPF service Argo (namespace dpf-operator-system) manages services on the DPUs. Keep their ownership separate.

NVIDIA architecture: https://networking-docs.nvidia.com/dpf/26.4.0/component-description

Step 1 | Download and edit the lab files

Run on the administration machine; keep the same Bash terminal for installation steps.

```
umask 077
LAB="$HOME/dpf-student-lab"
RAW=https://raw.githubusercontent.com/kirson-git/dsx-os/main
DOC="$RAW/DPF/user-documentation/dpf-only"
mkdir -p "$LAB"
curl -fsSL "$DOC/DPF-only-lab-files.zip" -o "$LAB/lab-files.zip"
unzip -n "$LAB/lab-files.zip" -d "$LAB"
kubectl get nodes
vi "$LAB/config.yaml"
vi "$LAB/30-hbn-application.yaml"
vi "$LAB/40-provisioning-application.yaml"
```

The ZIP contains every local file referenced by this guide. unzip -n preserves previous edits. Argo sources are pinned to repository revision 96b35f7. The settings below are lab examples: replace them for your site.

Setting	Asia	Europe
Tenant API VIP (manual)	192.168.0.203	192.168.0.200
DPU BMC / Redfish	192.168.0.249	192.168.0.246
DPU OS management (DHCP)	192.168.0.242	192.168.0.244
Ubuntu management SSH	root@192.168.0.232	ubuntu@192.168.0.231
Ubuntu HBN data address	10.0.0.1/24	10.0.0.2/24
HBN VRF / L3 VNI	ASIA / 100002	EUROPE / 100001

config.yaml: API 10.220.175.20:6443; VIP interface enp7s0; install-dhcp: "no". Existing lab DHCP is on l-ae-1137d (192.168.0.129). Reserve tenant VIPs outside DHCP. Each HBN network uses gateway 10.0.0.254.

30-hbn: edit hostname patterns, node selector, VRF, ASN, VNI and pools. 40-provisioning: edit device serials, BMC IPs/labels/selectors, node names, OPN/PSID, BFB and DNS. Keep matching references consistent. Passwords go in a private file in Step 5.

Step 2 | Install external Argo CD

Skip this installation if external Argo already exists with the repository health settings.

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update argo
helm install external-argo-cd argo/argo-cd \
  --version 9.4.1 --namespace argocd --create-namespace \
  --set configs.cm.installationID=dsx-platform \
  -f "$LAB/external-argo.yaml" --wait --timeout 10m
kubectl -n argocd get pods
```

Expected: Argo pods Running/Ready. The supplied values include CRD health handling and separate this Argo from DPF service Argo. The commands below sync through Kubernetes; an Argo CLI login or browser is not required.

Optional: open the Argo UI

```
kubectl -n argocd port-forward service/external-argo-cd-argocd-server 8080:443
# Keep this terminal open. Open https://localhost:8080 on this machine.
# In another terminal, retrieve the initial admin password locally:
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
```

Username: admin. If the initial Secret has been removed, use the existing administrator/SSO credentials. When running remotely, localhost belongs to that remote machine; use your existing tunnel if needed. Do not publish the password.

What is installed next?

Step 3 installs Local Path storage, cert-manager, Kamaji/etcd, Node Feature Discovery, Maintenance Operator, DPF and DPF service Argo. This bundle enables Local Path storage. For a cluster that already has storage, use the storage-selection guide before applying it.

Storage selection: https://github.com/kirson-git/dsx-os/blob/main/DPF/STAGED-INSTALL.md#2-choose-storage

Step 3 | Install prerequisites and DPF

Define this sync function once in the installation terminal; reuse it for the later stages.

```
dpf_sync() {
  kubectl -n argocd patch application "$1" --type merge -p \
'{"operation":{"sync":{"prune":false,"syncOptions":[
"ServerSideApply=true","ClientSideApplyMigration=false",
"FailOnSharedResource=true"],"syncStrategy":{"hook":{}}},
"retry":{"limit":5,"backoff":{"duration":"10s","factor":2,"maxDuration":"3m"}}}}'
}
kubectl apply -f "$LAB/00-platform.yaml"
dpf_sync dpf-platform
kubectl -n argocd get application dpf-platform \
  -o custom-columns=NAME:.metadata.name,SYNC:.status.sync.status,HEALTH:.status.health.status -w
```

Wait for Synced / Healthy. Ctrl+C stops watching only. Then confirm the latest sync operation succeeded:

```
kubectl -n argocd get application dpf-platform \
  -o jsonpath='{.status.operationState.phase}{"\n"}{.status.operationState.message}{"\n"}'
kubectl -n dpf-operator-system get pods
kubectl get storageclasses
kubectl get pvc -A
```

Expected: Succeeded, required controllers ready, and their PVCs Bound. If a sync fails, inspect its message before continuing. Server-side apply avoids the large-CRD client annotation limit.

Use only these staged Applications for this walkthrough. Do not also apply the combined install.yaml or the older dpf-operator-config / dpf-tenants Applications; they would overlap ownership.

Steps 4-5 | Create tenants and validate BMC access

A tenant control plane is created before its DPU worker joins.

4. Apply your operator and tenant settings

```
kubectl apply -f "$LAB/config.yaml"
kubectl -n argocd get application dpf-site -w
# Ctrl+C after Synced / Healthy, then:
kubectl get dpuclusters -A
kubectl -n tenant-asia get pods
kubectl -n tenant-europe get pods
```

Expected: tenant-asia and tenant-europe Ready. config.yaml starts auto-sync. It configures the operator and tenants; HBN and hardware use the next two files.

5. Create a private credential file

```
install -d -m 700 "$HOME/.config/dpf-lab"
touch "$HOME/.config/dpf-lab/bmc-credentials"
chmod 600 "$HOME/.config/dpf-lab/bmc-credentials"
vi "$HOME/.config/dpf-lab/bmc-credentials"
```

Enter exactly these two lines, replacing the password; no quotes. Keep the file outside Git:

```
username=root
password=YOUR_BMC_PASSWORD
```

```
bash "$LAB/credentials.sh" --ip 192.168.0.246,192.168.0.249 \
  --bmc-pass "$HOME/.config/dpf-lab/bmc-credentials" \
  --node l-ae-1137d-001 --insecure-bmc
kubectl -n dpf-operator-system get secret bmc-shared-password
```

The helper tests both Redfish logins from a Kubernetes pod before saving the Secret. --insecure-bmc is only for this self-signed lab. The BMC password is separate from Ubuntu/DPU SSH credentials.

Steps 6-7 | Prepare HBN and provision the DPUs

HBN definitions are prepared first; containers start after DPU OS installation and tenant join.

6. Apply HBN networking definitions

```
kubectl apply -f "$LAB/30-hbn-application.yaml"
dpf_sync dpf-hbn
kubectl -n argocd get application dpf-hbn -w
# Ctrl+C after Synced, then check the operation:
kubectl -n argocd get application dpf-hbn \
  -o jsonpath='{.status.operationState.phase}{"\n"}'
```

Expected: Succeeded. The YAML defines HBN services, interfaces, service chains and tenant-scoped IP pools. The two 10.0.0.0/24 networks remain isolated by their separate tenant routing domains.

7. Register hardware and request provisioning

```
kubectl apply -f "$LAB/40-provisioning-application.yaml"
dpf_sync dpf-provisioning
kubectl -n argocd get application dpf-provisioning -w
# Ctrl+C after sync; watch the actual devices:
kubectl -n dpf-operator-system get dpus -w
```

This can reimage the DPUs. DPUDevice/DPUNode supply identities and BMC endpoints. DPUSet/DPUDeployment select the tenant, BFB and DPUFlavor. The BFB supplies the DPU OS and bundled DOCA software; the flavor requests the lab firmware and zero-trust settings.

Argo Synced means the manifests were applied. It does not prove the DPU booted, HBN started, or host traffic works. The staged profile uses external reboot handling and maintenance holds; follow the next page when paused. This does not install Ubuntu on the x86 server.

Provisioning checkpoints and logs

Inspect the reason before releasing a hold or rebooting hardware.

```
kubectl -n dpf-operator-system describe dpus
kubectl -n dpf-operator-system get dpunodemaintenances
kubectl -n dpf-operator-system get events --sort-by=.lastTimestamp
```

If the phase is Node Effect

The lab profile deliberately holds maintenance. After the affected host workload is stopped/drained, read the maintenance object, then acknowledge only that hold. Replace HOLD_NAME with the name printed above:

```
HOLD=HOLD_NAME
kubectl -n dpf-operator-system describe dpunodemaintenance "$HOLD"
kubectl -n dpf-operator-system annotate dpunodemaintenance "$HOLD" \
  provisioning.dpu.nvidia.com/wait-for-external-nodeeffect=false --overwrite
```

If Rebooted reports WaitingForManualPowerCycleOrReboot

Use the approved server BMC/console procedure to shut down and power-cycle the affected physical host/DPU. A reboot of the administration machine is not the required action. Wait for the host to return, then inspect DPU conditions again. Do not reapply YAML repeatedly or bypass conditions to make a status look ready.

Watch four log panes

```
# Requires tmux on the administration machine.
bash "$LAB/monitor.sh"
# Detach: Ctrl+b, then d. Reopen:
tmux attach -t dpf-monitor
```

The monitor shows DPU/BFB status, controller logs and events. Logs are saved under $HOME/dpf-logs/TIMESTAMP. Proceed when DPU conditions and tenant worker nodes are ready, then verify HBN pods in Steps 10-11.

Step 8 | SSH to the Ubuntu server

SSH uses the management network. 10.0.0.1 is the Asia server data address behind HBN.

Use the existing management network and server credentials. This helper pod runs on a Kubernetes node that can reach the server; change nodeName in access-pod.yaml for another site.

```
LAB="$HOME/dpf-student-lab"
vi "$LAB/access-pod.yaml"
# Recreate only this temporary helper (also works after it has expired):
kubectl -n dpf-operator-system delete pod dpf-hbn-recovery-access \
  --ignore-not-found --wait=true --timeout=60s
kubectl apply -f "$LAB/access-pod.yaml"
kubectl -n dpf-operator-system wait pod/dpf-hbn-recovery-access \
  --for=condition=Ready --timeout=120s
ssh -o \
  'ProxyCommand=kubectl -n dpf-operator-system exec -i dpf-hbn-recovery-access -- nc %h %p' \
  root@192.168.0.232
```

Expected: an Ubuntu shell on the Asia x86 server. Verify its SSH fingerprint through your administrator, then use your server key/password. The helper expires after four hours. No password is included in the PDF or ZIP.

Inside Asia Ubuntu

```
hostname
ip -br -4 address
ip -4 route
DATA_IF=$(ip -o -4 addr show | awk '$4=="10.0.0.1/24" {print $2; exit}')
if [ -n "$DATA_IF" ]; then ip -4 addr show dev "$DATA_IF"; fi
```

Expected: 10.0.0.1/24 on the DPU-facing host interface; default route via 10.0.0.254. If DATA_IF is empty, stop: host netplan, DPU service or links are not ready. This guide retains the existing host configuration; Argo does not assign that host address.

Step 9 | Demonstrate traffic through HBN

Run these commands inside Asia Ubuntu, after confirming DATA_IF in Step 8.

```
ip -4 route get 1.1.1.1 from 10.0.0.1
ping -4 -I "$DATA_IF" -c 3 -W 2 10.0.0.254
getent ahostsv4 cisco.com
ping -4 -I "$DATA_IF" -c 3 -W 2 cisco.com
curl -4 --noproxy '*' --interface "$DATA_IF" \
  --connect-timeout 5 --max-time 15 -sS -o /dev/null \
  -w 'HTTP=%{http_code} source=%{local_ip}\n' https://www.cisco.com
```

Expected: the route uses the DPU-facing interface and 10.0.0.254. DNS resolves. HTTPS succeeds and curl prints source=10.0.0.1 (before upstream NAT). Public sites may filter ICMP; use HTTPS as a separate connectivity check.

Show the tenant boundary

```
# From Asia, attempt to reach the Europe data address:
ping -4 -I "$DATA_IF" -c 3 -W 2 10.0.0.2
```

Expected for the isolated lab: no replies. Only present this as evidence of isolation after both servers have their data interfaces and successful egress. A down Europe host is not an isolation test. This ICMP check is a demonstration, not a complete security audit.

Open Europe in a second administration terminal

```
ssh -o \
  'ProxyCommand=kubectl -n dpf-operator-system exec -i dpf-hbn-recovery-access -- nc %h %p' \
  ubuntu@192.168.0.231
# Inside Europe:
DATA_IF=$(ip -o -4 addr show | awk '$4=="10.0.0.2/24" {print $2; exit}')
ping -4 -I "$DATA_IF" -c 3 -W 2 cisco.com
ping -4 -I "$DATA_IF" -c 3 -W 2 10.0.0.1
```

Expected: Europe data address 10.0.0.2, working egress, no cross-tenant reply. Repeat the HTTPS command above from Europe; its source must be 10.0.0.2.

Step 10 | Open the Asia tenant Kubernetes API

HBN runs on the DPU inside its tenant cluster, not inside the Ubuntu server.

Each tenant has its own kubeconfig. Run these commands on the administration machine; the file grants tenant access and must remain private. The service port is discovered dynamically.

```
umask 077
KCFG="$HOME/.config/dpf-lab/tenant-asia.kubeconfig"
mkdir -p "$HOME/.config/dpf-lab"
kubectl -n tenant-asia get secret tenant-asia-admin-kubeconfig \
  -o jsonpath='{.data.admin\.conf}' | base64 -d > "$KCFG"
chmod 600 "$KCFG"
CLUSTER=$(kubectl --kubeconfig "$KCFG" config view -o jsonpath='{.clusters[0].name}')
SERVER=$(kubectl --kubeconfig "$KCFG" config view -o jsonpath='{.clusters[0].cluster.server}')
TLS_NAME=$(python3 -c 'import sys,urllib.parse; print(urllib.parse.urlsplit(sys.argv[1]).hostname)' "$SERVER")
kubectl --kubeconfig "$KCFG" config set-cluster "$CLUSTER" \
  --server=https://127.0.0.1:16443 --tls-server-name="$TLS_NAME"
PORT=$(kubectl -n tenant-asia get service tenant-asia -o jsonpath='{.spec.ports[0].port}')
kubectl -n tenant-asia port-forward service/tenant-asia "16443:$PORT"
```

Leave this terminal open. The local kubeconfig retains its CA and uses the original API hostname for TLS verification; it points to the localhost tunnel only for transport. No insecure-skip-tls-verify flag is needed.

In another administration terminal

```
KCFG="$HOME/.config/dpf-lab/tenant-asia.kubeconfig"
kubectl --kubeconfig "$KCFG" get nodes -o wide
kubectl --kubeconfig "$KCFG" -n dpf-operator-system \
  get pods -l app.kubernetes.io/name=hbn -o wide
```

Expected: Asia DPU node Ready and its HBN pod fully Ready/Running. If there are no worker nodes or HBN pods, installation is not complete; return to DPU conditions and service-controller logs.

Step 11 | Inspect HBN, VRF and BGP

Continue in the second administration terminal, with the tenant API tunnel running.

```
HBN=$(kubectl --kubeconfig "$KCFG" -n dpf-operator-system \
  get pods -l app.kubernetes.io/name=hbn -o jsonpath='{.items[0].metadata.name}')
kubectl --kubeconfig "$KCFG" -n dpf-operator-system get pod "$HBN" \
  -o jsonpath='{range .spec.containers[*]}{.name}{"  "}{.image}{"\n"}{end}'
# Set the container name whose image is nvcr.io/nvidia/doca/doca_hbn:
HBN_CONTAINER=REPLACE_WITH_HBN_CONTAINER_NAME
kubectl --kubeconfig "$KCFG" -n dpf-operator-system \
  exec -it "$HBN" -c "$HBN_CONTAINER" -- bash
```

Inside the HBN container

```
nv show vrf
ip -4 route show vrf ASIA
vtysh -c 'show bgp summary'
vtysh -c 'show bgp l2vpn evpn summary'
exit
```

Expected configuration: VRF ASIA, ASN 65201 and L3 VNI 100002. The host-facing HBN network is 10.0.0.0/24. BGP/EVPN peers must be established for the lab fabric. The exact peer addresses and route counts depend on that fabric.

Back on the administration machine: collect HBN logs

```
kubectl --kubeconfig "$KCFG" -n dpf-operator-system \
  logs "$HBN" --all-containers --tail=100
```

The BFB/flavor configure the DPU OS and zero-trust mode. HBN configures the tenant data path. A failed cross-tenant ping alone does not prove hardware zero-trust mode. NVIDIA DPUDeployment reference: https://networking-docs.nvidia.com/dpf/26.4.0/dpudeployment

Validation scope: YAMLs match the saved lab Argo sources; commands/layout were checked. This document is not a claim that a fresh reinstall passed. On 2026-10-19 the live Argo apps were Synced/Healthy, while DPU resources awaited manual reboot and the Asia tenant had no worker nodes.